Встреча 3

Разработка микросервисных приложений на Golang

YADRO

Часто встречающиеся ошибки и замечания

Компиляция регекса внутри функции

```
func DoSomething(s string) {
  re := regex.MustCompile(“xx”)
  words := re.FindAllString(s)
```

```
var re = regex.MustCompile(“xx”)

func DoSomething(s string) {
  words := re.FindAllString(s)
```

А нужен ли регекс?

```
// Библиотеки strings + unicode дают нам все нужное


words := strings.FieldsFunc(
    phrase,
    func(r rune) bool {
        return !unicode.IsDigit(r) && !unicode.IsLetter(r)
    },
)
```

Уникальное множество

```
unique := make(
        map[string]struct{})

if _, ok := unique[word]; !ok {
    unique[word] = struct{}
}
```

```
unique := make(map[string]bool)


if unique[word] {
    unique[word] = true
}
```

Сборка ключей или значений map -> slice

```
keys := make([]string, 0, len(unique))
for k := range unique {
    keys = append(keys, k)
}
            // vs
keys := slices.Collect(maps.Keys(unique))
```

Стоп-слова

Варианты:

- отдельный файл -> map[string]bool на старте

Стоп-слова

Варианты:

- отдельный файл -> map[string]bool на старте
- map[string]bool в коде

```
stopWords := map[string]bool{ “a”: true, “the”: true}
```

Стоп-слова

Варианты:

- отдельный файл -> map[string]bool на старте
- map[string]bool в коде
- большой switch case

```
switch word {
    case “a”, “the”, “and”, … :
        return true
}
```

Стоп-слова

Варианты:

- отдельный файл -> map[string]bool на старте
- map[string]bool в коде
- большой switch case

```
switch word {
    case “a”, “the”, “and”, … :
        return true
}
```



Стоп-слова

…

в библиотеке snowball есть функция проверки стоп слов



Пакет words 😁

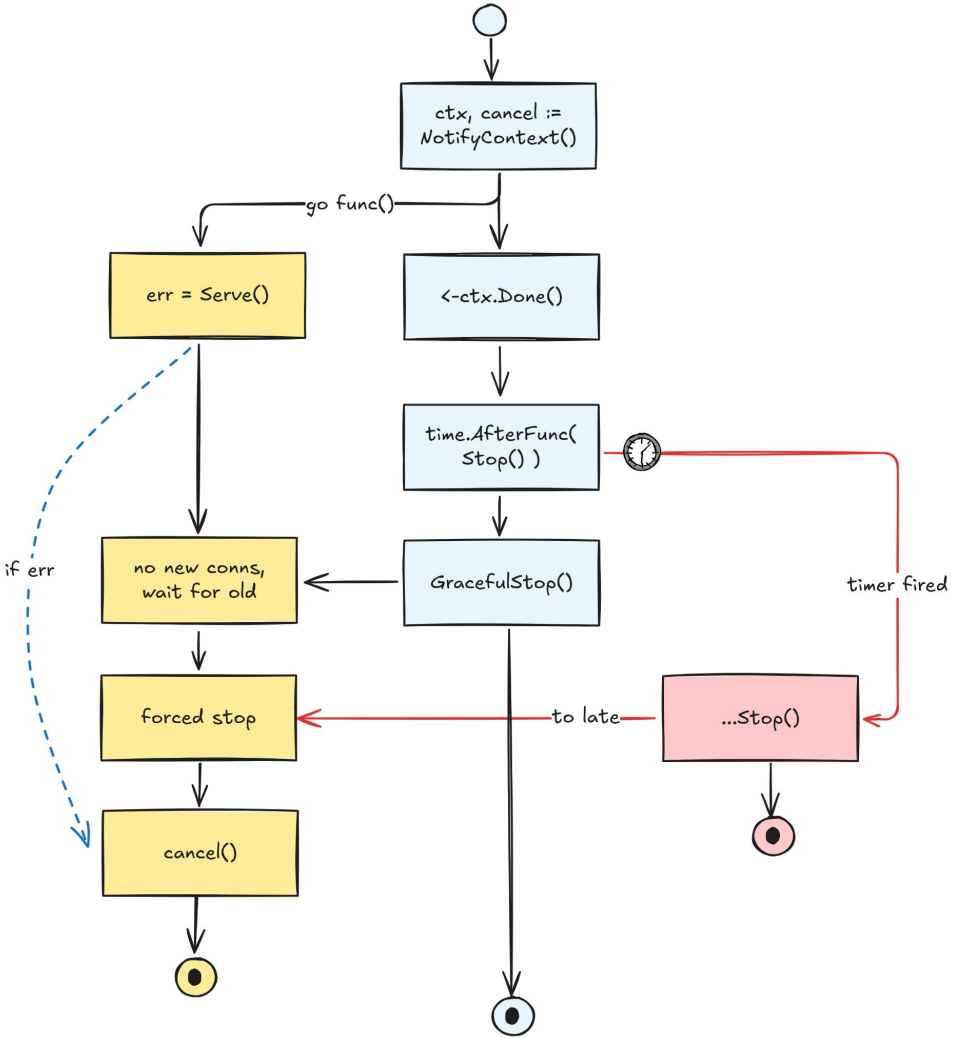
Критерии приемки

1. Микросервис компилируется в docker image, запускается через compose файл и проходит тесты.
2. Используются библиотеки petname и snowball.
3. Используется библиотека snowball, код логики нормализации находится в папке search-services/words/words.
4. Сервер конфигурируется через cleanenv пакет и должен уметь запускаться как с config.yaml файлом через флаг -config, так и через переменные среды, в этом задании - PETNAME_GRPC_PORT, WORDS_GRPC_PORT
5. Используется golang 1.25+

for i := 0; i < int(in.Names); i++ {

- int(int64) - не самая лучшая стратегия 💇
- i := int64(0)
- for range in.Names { … // e.g. for range 5

Graceful Shutdown, второй сезон - gRPC



Shiny Object Syndrome (синдром блестящего объекта)

склонность людей и организаций отвлекаться на новые, привлекательные идеи, технологии или тренды, часто в ущерб основным стратегическим целям.

Основные характеристики

- Метафора синдрома основана на поведении сороки: **отвлекается на блестящие предметы**.
- Проявления включают постоянное переключение между проектами, незавершённые задачи и отсутствие фокуса.
- Последствия: потеря ресурсов, упущенные возможности и провал в достижении целей.

Причины возникновения

- Нечеткие цели, отсутствие фокуса.
- Страх упустить возможности (FOMO).
- Неуверенность в текущих методах работы.
- Желание быстрого роста и инноваций.

Go Project Layout

1. Flat
    a. 5-7 файлов
    b. особенно пригодно для библиотек
    c. идеально для words microservice
    d. вероятны конфликты имен

Go Project Layout

1. Flat
2. Package
    a. отделяем мух от котлет
    b. аналогично тому, как мы структурируем информацию по папкам ФС
    c. пакеты можно отдавать другим проектам и выделять в отдельные репозитории
    d. 10+ файлов
    e. возможны циклические зависимости, от которых сложно избавиться

Go Project Layout

1. Flat
2. Package
3. Layers
   a. слои зависят друг от друга в одном направлении, например, server -> service -> db
   b. SOLID, верхние слои зависят от абстракции, а не нижних слоев
   c. понятная логика расположения кода

Go Project Layout

1. Flat
2. Package
3. Layers
4. Hexagonal
    a. отец-основатель ЧА, DDD, XXX
    b. идеально для микросервисов
    c. ее мы и постараемся использовать в нашем проекте

Гексагональная Архитектура (Порты и Адаптеры)

- Архитектурный шаблон (2005й год)

Гексагональная Архитектура (Порты и Адаптеры)

- Архитектурный шаблон
- Основная цель - отделить логику приложения от используемых сервисов

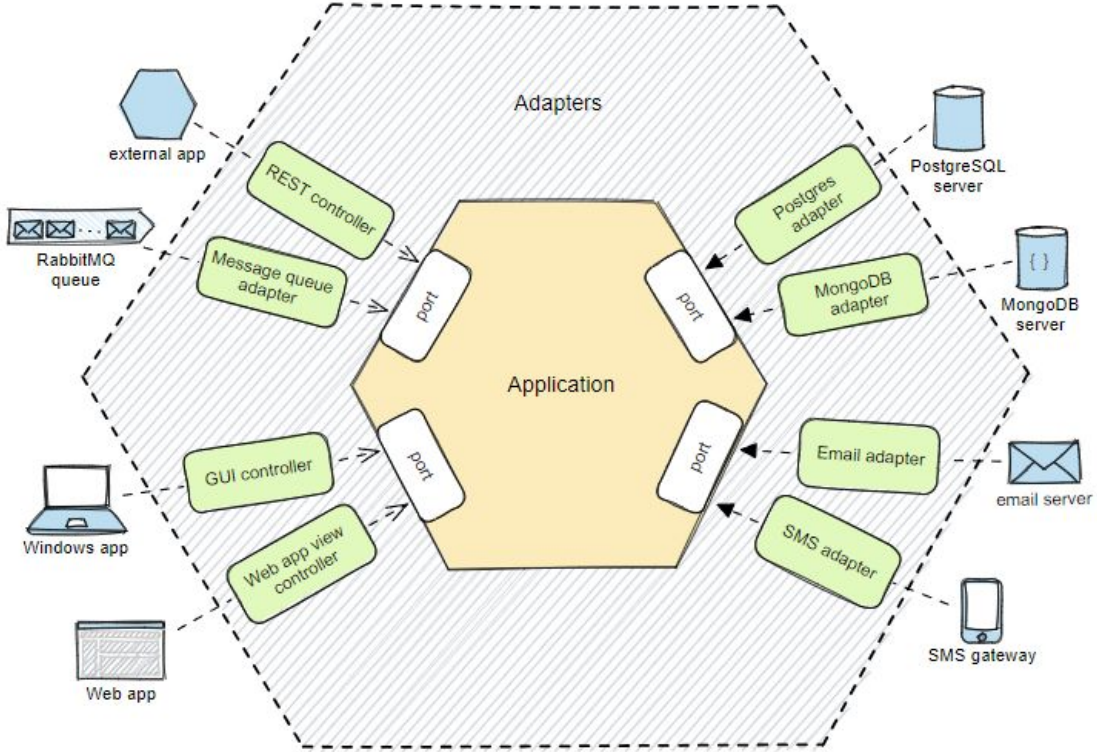
Гексагональная Архитектура (Порты и Адаптеры)

- Архитектурный шаблон
- Основная цель - отделить логику приложения от используемых сервисов
- Три части проекта
  - Основная логика (Core) - ничего не знает о внешнем мире - db, http, etc
  - Порты (Ports) - интерфейсы, которые нужны логике (SOLID)
  - Адаптеры (Adapters) - адаптируют внешнюю среду под основную логику, имплементируя порты. Это DB, gRPC, REST, timers/clocks и подобные.

Гексагональная Архитектура (Порты и Адаптеры)

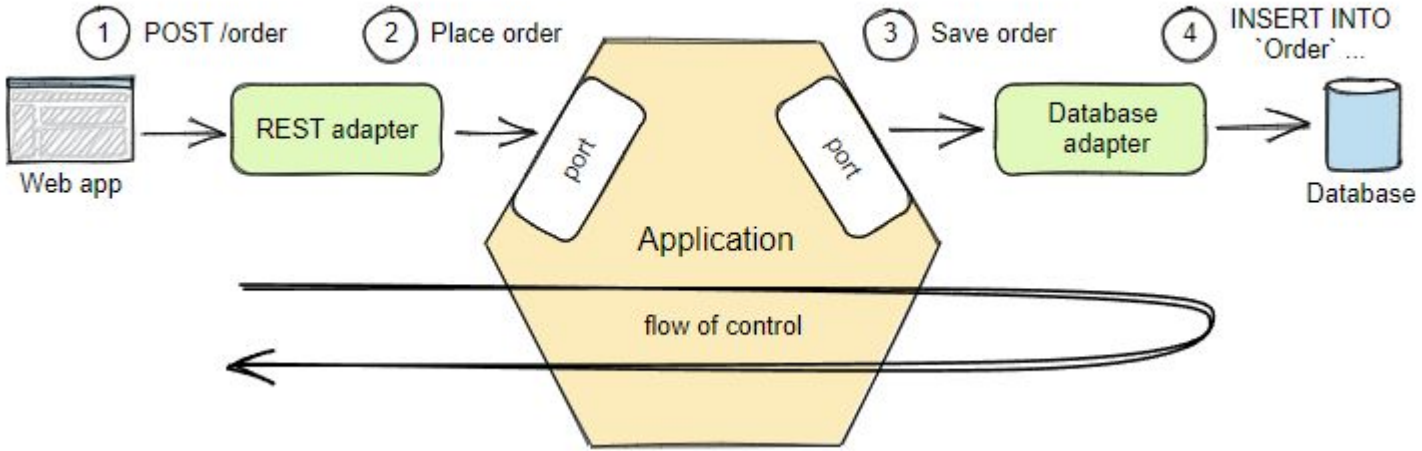
- Архитектурный шаблон
- Основная цель - отделить логику приложения от используемых сервисов
- Три части проекта
  - Основная логика (Core) - ничего не знает о внешнем мире - db, http, etc
  - Порты (Ports) - интерфейсы, которые нужны логике (SOLID)
  - Адаптеры (Adapters) - адаптируют внешнюю среду под основную логику, имплементируя порты. Это DB, gRPC, REST, timers/clocks и подобные.
- Преимущества
  - Модульность
  - Сопровождаемость
  - Тестируемость
  - Масштабируемость

Гексагональная Архитектура (Порты и Адаптеры)



wiki

Гексагональная Архитектура (Порты и Адаптеры)



wiki

Гексагональная Архитектура (Порты и Адаптеры)

